

IO_PWM.h File Reference

IO Driver functions for PWM channels. [More...](#)

```
#include "IO_Driver.h"
```

Data Structures

struct **IO_PWM_SAFETY_CONF**
Safety configuration for the PWM outputs. [More...](#)

struct **IO_PWM_CURRENT_QUEUE**
PWM current measurement queue. [More...](#)

Macros

Size of current queue

Size of the queue holding the measured values of the equidistant current measurement

```
#define IO_PWM_CURRENT_QUEUE_MAX 6U
```

Functions

IO_ErrorType **IO_PWM_Init** (**ubyte1** pwm_channel, **ubyte2** frequency, **bool** polarity, **bool** diag_margin, **IO_PWM_SAFETY_CONF** const *const safety_conf)
Setup a single PWM output. [More...](#)

IO_ErrorType **IO_PWM_InitWithLowside** (**ubyte1** pwm_channel, **ubyte2** frequency, **bool** polarity, **bool** diag_margin, **IO_PWM_SAFETY_CONF** const *const safety_conf)
Setup a single PWM output (which can be connected to a low side channel)
[More...](#)

IO_ErrorType **IO_PWM_DeInit** (**ubyte1** pwm_channel)
Deinitializes a PWM output. [More...](#)

IO_ErrorType **IO_PWM_SetDuty** (**ubyte1** pwm_channel, **ubyte2** duty_cycle, **ubyte2** *const high_time_fb, **ubyte2** *const period_fb)
Set the duty cycle for a PWM channel. [More...](#)

IO_ErrorType **IO_PWM_GetCur** (**ubyte1** pwm_channel, **ubyte2** *const current, **bool** *const fresh)
Returns the measured current of the given channel. [More...](#)

IO_ErrorType **IO_PWM_GetCurQueue** (**ubyte1** pwm_channel, **IO_PWM_CURRENT_QUEUE** *const current_queue)
Returns the measured current values since the last call of the given channel.
[More...](#)

IO_ErrorType **IO_PWM_ResetProtection** (**ubyte1** pwm_channel, **ubyte1** *const reset_cnt)

Reset the output protection for a PWM channel. [More...](#)

IO_ErrorType **IO_PWM_ResolveOpenLoadShortCircuit** (**ubyte1** pwm_channel)

Resolve an open load or short circuit to battery voltage condition for a PWM channel. [More...](#)

Detailed Description

IO Driver functions for PWM channels.

Contains all service functions for the PWM (Pulse Width Modulation). Up to 36 channels can be configured: [IO_PWM_00](#) .. [IO_PWM_35](#)

- All PWM outputs have timer feedback
- All PWM outputs have current measurement
- The PWM outputs will be switched off immediately if the continuous current is above 4A. The step function for retrieving the current is [IO_PWM_GetCur\(\)](#) or [IO_PWM_GetCurQueue\(\)](#).

PWM-API Usage:

- [Examples for PWM API functions](#)

PWM output protection

Each PWM output is individually protected against overcurrent. Whenever a fatal overcurrent situation (i.e., a current greater than 5.5A) is detected on any safe or unsafe PWM output, the output is disabled by software latest within 7ms. Note that disabled by software means that the duty cycle of the output signal is set to 0% but the states of the safety switch and PWM power stage remain unchanged.

When entering the protection state, the PWM output has to remain in this state for at least 10s. After this wait time the PWM output can be reenabled via function [IO_PWM_ResetProtection\(\)](#). Note that the number of reenabling operations for a single PWM output is limited to 10. Refer to function [IO_PWM_ResetProtection\(\)](#) for more information on how to reenable a PWM output.

PWM code examples

Examples for using the PWM API

PWM initialization example

```
// Setup a PWM output
IO_PWM_Init(IO_PWM_00,
            100,          // frequency is 100 Hz
            TRUE,         // positive polarity
            FALSE,        // no diag margin
            NULL);        // not safety critical

// Setup a PWM output
```

```

IO_PWM_Init(IO_PWM_01,
            166,          // frequency is 166 Hz
            TRUE,         // positive polarity
            FALSE,        // no diag margin
            NULL);        // not safety critical

// Setup a PWM output
IO_PWM_Init(IO_PWM_12,
            200,          // frequency is 200 Hz
            TRUE,         // positive polarity
            FALSE,        // no diag margin
            NULL);        // not safety critical

```

PWM task example

```

ubyte2 curr;
bool new;

IO_PWM_SetDuty(IO_PWM_00,
               0x8000,    // set duty cycle to 50%
               NULL,      // feedback high time measurement ignored
               NULL);     // feedback period measurement ignored

IO_PWM_GetCur(IO_PWM_00, // read current value of PWM output
               &curr,     // variable to store current
               &new);     // variable to store information if a current
                           // value is available

```

Macro Definition Documentation

```
#define IO_PWM_CURRENT_QUEUE_MAX 6U
```

maximum number of items in the current queue

Function Documentation

IO_ErrorType IO_PWM_DeInit (ubyte1 pwm_channel)

Deinitializes a PWM output.

Allows the re-initialization of the output by other functions

Parameters

pwm_channel PWM channel, one of:

- **IO_PWM_00 .. IO_PWM_35**

Returns

IO_ErrorType

Return values

IO_E_OK	everything fine
IO_E_INVALID_CHANNEL_ID	the given channel id does not exist
IO_E_CHANNEL_NOT_CONFIGURED	the given channel is not configured
IO_E_CH_CAPABILITY	the given channel has no PWM capability
IO_E_DRIVER_NOT_INITIALIZED	the common driver init function has not been called before
IO_E_UNKNOWN	an unknown error occurred


```
IO_ErrorType IO_PWM_GetCur ( uint8_t      pwm_channel,
                             uint16_t *const current,
                             bool *const  fresh
                           )
```

Returns the measured current of the given channel.

Parameters

pwm_channel PWM channel, one of:

- `IO_PWM_00` .. `IO_PWM_35`

[out] **current** Measured current in mA Range: 0..7500 (0A .. 7.500A)

[out] **fresh** Indicates if new values are available since the last call.

- **TRUE**: Value in "current" is valid
- **FALSE**: No new value available.

Returns

`IO_ErrorType`

Return values

<code>IO_E_OK</code>	everything fine
<code>IO_E_CHANNEL_NOT_CONFIGURED</code>	the given channel is not configured
<code>IO_E_INVALID_CHANNEL_ID</code>	the given channel id does not exist
<code>IO_E_CH_CAPABILITY</code>	the given channel is not a PWM channel
<code>IO_E_CM_CALIBRATION</code>	the zero current calibration failed
<code>IO_E_NULL_POINTER</code>	a NULL pointer has been passed to the function
<code>IO_E_DRIVER_NOT_INITIALIZED</code>	the common driver init function has not been called before
<code>IO_E_REFERENCE</code>	the internal reference voltage is out of range
<code>IO_E_UNKNOWN</code>	an unknown error occurred

Attention

The current measurement is equidistant which means that the sampling happens synchronous to the PWM period. Every 1ms a current value will be captured. The captured current values will be averaged over the time of one period of the PWM signal and then provided to the application.

Remarks

- When the function `IO_PWM_GetCur()` is called, the internal queue holding the values of the current measurement is flushed. If the function is called more than once

in a cycle it may or may not deliver new values, depending on how many values the equidistant current measurement has sampled since the last call.

- If there is no new current value available (for example the function `IO_PWM_GetCur()` gets called more frequently than the PWM period) the flag `fresh` will be set to `FALSE`.
- If the functions `IO_PWM_GetCurQueue()` and `IO_PWM_GetCur()` are called after each other, only the first function will deliver a current value because both functions will empty the internal measurement queue.


```
IO_ErrorType IO_PWM_GetCurQueue ( ubyte1 pwm_channel,
                                   IO_PWM_CURRENT_QUEUE *const current_queue
                                   )
```

Returns the measured current values since the last call of the given channel.

Parameters

pwm_channel PWM channel, one of:

- `IO_PWM_00 .. IO_PWM_35`

[out] **current_queue** Queue holding the current values since the last call of the step function including a queue overrun flag and a counter for the quantity of available values

Returns

`IO_ErrorType`

Return values

<code>IO_E_OK</code>	everything fine
<code>IO_E_CHANNEL_NOT_CONFIGURED</code>	the given channel is not configured
<code>IO_E_INVALID_CHANNEL_ID</code>	the given channel id does not exist
<code>IO_E_CH_CAPABILITY</code>	the given channel is not a PWM channel
<code>IO_E_CM_CALIBRATION</code>	the zero current calibration failed
<code>IO_E_NULL_POINTER</code>	a NULL pointer has been passed to the function
<code>IO_E_DRIVER_NOT_INITIALIZED</code>	the common driver init function has not been called before
<code>IO_E_REFERENCE</code>	the internal reference voltage is out of range
<code>IO_E_UNKNOWN</code>	an unknown error occurred

Attention

The current measurement is equidistant which means that the sampling happens synchronous to the PWM period. Every 1ms a current value will be captured. The captured current values will be averaged over the time of one period of the PWM signal and then provided to the application in a queue. The queue storage was chosen to avoid the loss of any measurement value if the user application runs asynchronous to the current measurement or if its cycle time is lower than the PWM period. (Size of queue: `IO_PWM_CURRENT_QUEUE_MAX`, Queue data type: `IO_PWM_CURRENT_QUEUE`)

Remarks

- When the function `IO_PWM_GetCurQueue()` is called, the internal queue holding the values of the current measurement is flushed. If the function is called more than

once in a cycle it may or may not deliver new values, depending on how many values the equidistant current measurement has sampled since the last call.

- If there is no new current value available (for example the function `IO_PWM_GetCurQueue()` gets called more frequently than the PWM period) the value `count` in the structure `current_queue` will be 0.
- If the functions `IO_PWM_GetCurQueue()` and `IO_PWM_GetCur()` are called after each other, only the first function will deliver a current value because both functions will empty the internal measurement queue.


```
IO_ErrorType IO_PWM_Init ( ubyte1                                pwm_channel,
                           ubyte2                                frequency,
                           bool                                  polarity,
                           bool                                  diag_margin,
                           IO_PWM_SAFETY_CONF const *const     safety_conf
                           )
```

Setup a single PWM output.

Parameters

pwm_channel PWM channel, one of:

- `IO_PWM_00 .. IO_PWM_35`

frequency PWM frequency (50Hz .. 1000Hz, only predefined frequencies with a period of an integral multiple of 1ms, 0.5ms or 0.25ms are possible)

polarity Polarity of output signal

- `FALSE`: Low output signal is variable
- `TRUE`: High output signal is variable

diag_margin Indicate if a margin should be applied or not.

- `TRUE`: margin is on
- `FALSE`: no margin will be applied
- If a PWM channel is configured safety relevant this parameter has to be set to `TRUE`

[in] **safety_conf** Safety configuration of the PWM channel. The `low_side_channel` field is evaluated only with `IO_PWM_InitWithLowside()`. If a safety configuration is passed, the parameter `diag_margin` will be forced to `TRUE`.

Returns

`IO_ErrorType`

Return values

<code>IO_E_OK</code>	everything fine
<code>IO_E_CHANNEL_BUSY</code>	the PWM output channel or the timer input channel is currently used by another function
<code>IO_E_INVALID_CHANNEL_ID</code>	the given channel id does not exist or is not available on the used ECU variant
<code>IO_E_CH_CAPABILITY</code>	

IO_E_INVALID_PARAMETER

the given channel is not a PWM channel or the used ECU variant does not support this function

IO_E_CM_CALIBRATION

a given parameter is out of range

IO_E_DRIVER_NOT_INITIALIZED

the zero current calibration failed

IO_E_FPGA_NOT_INITIALIZED

the common driver init function has not been called before

IO_E_INVALID_SAFETY_CONFIG

the FPGA has not been initialized

IO_E_DRV_SAFETY_CONF_NOT_CONFIG

an invalid safety configuration has been passed

IO_E_DRV_SAFETY_CYCLE_RUNNING

global safety configuration is missing

the init function was called after the task begin function

IO_E_UNKNOWN

an unknown error occurred

Remarks

- For the equidistant current measurement to work properly only certain frequencies are supported:

Available frequencies:		
Period[ms]	Frequency[Hz]	Current sample period[ms]
1	1000	1
1.25	800	5
1.5	666 (666.6666)	3
1.75	571 (571.4285)	7
2	500	2
2.25	444 (444.4444)	9
2.5	400	5
2.75	363 (363.6363)	11
3	333 (333.3333)	3
3.25	307 (307.6923)	13
3.5	285 (285.7142)	7
3.75	266 (266.6666)	15
4	250	4
4.25	235 (235.2941)	17
4.5	222 (222.2222)	9
4.75	210 (210.5263)	19

5	200	5
5.5	181 (181.8181)	11
6	166 (166.6666)	6
6.5	153 (153.8461)	13
7	142 (142.8571)	7
7.5	133 (133.3333)	15
8	125	8
8.5	117 (117.6470)	17
9	111 (111.1111)	9
9.5	105 (105.2631)	19
10	100	10
11	90 (90.9090)	11
12	83 (83.3333)	12
13	76 (76.9230)	13
14	71 (71.4285)	14
15	66 (66.6666)	15
16	62 (62.5)	16
17	58 (58.8235)	17
18	55 (55.5555)	18
19	52 (52.6315)	19
20	50	20

- If you select an unavailable frequency within the allowed range, the next higher available frequency will be used. For example, if 180 Hz is specified, 181.8 Hz will be used.
- The associated timer loopback channel will also be configured for open load and short circuit detection.
- The associated current measurement will also be configured.
- The duty cycle cannot exceed the margin of 100us(lower boundary) and 200us (upper boundary) used for diagnostic if the parameter diag_margin is **TRUE**. This mode is important for hydraulic coils. If the parameter diag_margin is **FALSE**, no duty cycle range margin will be applied.
- All PWM channels have their own frequency time base.
- If `safety_conf != NULL`, the internal checker modules check the given channels against the `current` parameter in `safety_conf`, and the period and duty cycle

feedback against the output. For more details about the current checking refer to the definition of `IO_PWM_SAFETY_CONF`.

- If `safety_conf != NULL`, the parameter `diag_margin` is forced to `TRUE` to allow diagnostics
- Static friction and stiction can cause a hysteresis and make the control of a hydraulic valve erratic and unpredictable. In order to counteract these hysteresis effects, small vibrations about the desired position shall be created in the valve. This constantly breaks the static friction ensuring that it will move even with small input changes, and the effects of hysteresis are average out. A proper setting of PWM frequency according to the resonance frequency of the actuator allows to adjust this desired small vibration, low enough in amplitude to prevent noticeable oscillations on the hydraulic output but sufficient high to prevent friction. The PWM frequency can be set in the range of 50 .. 1000Hz, a typical range for hydraulic valves to operate without friction is 90 .. 160Hz.

IO_ErrorType

```

IO_PWM_InitWithLowside      ( ubyte1          pwm_channel,
                             ubyte2          frequency,
                             bool            polarity,
                             bool            diag_margin,
                             IO_PWM_SAFETY_CONF const *const safety_conf
                             )

```

Setup a single PWM output (which can be connected to a low side channel)

Parameters

pwm_channel PWM channel, one of:

- `IO_PWM_00 .. IO_PWM_35`

frequency PWM frequency (50Hz .. 1000Hz, only predefined frequencies with a period of an integral multiple of 1ms, 0.5ms or 0.25ms are possible)

polarity Polarity of output signal

- `FALSE`: Low output signal is variable
- `TRUE`: High output signal is variable

diag_margin Indicate if a margin should be applied or not.

- `TRUE`: margin is on
- `FALSE`: no margin will be applied
- If a PWM channel is configured safety relevant this parameter has to be set to `TRUE`

[in] **safety_conf** Relevant safety configuration of the PWM channel. If a safety configuration is passed the parameter `diag_margin` will be forced to `TRUE`.

Returns

IO_ErrorType

Return values

The return values match exactly the **return values of IO_PWM_Init()**.

Remarks

- Unlike with `IO_PWM_Init()`, the `low_side_channel` field of the safety configuration is evaluated with this function.

- If configured, the low side switch of a PWM channel is switched on and off together with the high side safety switch of the channel.
- The **remarks for IO_PWM_Init()** also apply here.


```
IO_ErrorType IO_PWM_ResetProtection ( ubyte1      pwm_channel,
                                     ubyte1 *const reset_cnt
                                     )
```

Reset the output protection for a PWM channel.

Parameters

pwm_channel PWM channel, one of:

- `IO_PWM_00 .. IO_PWM_35`

[out] **reset_cnt** Protection reset counter. Indicates how often the application already reset the protection.

Returns

`IO_ErrorType`

Return values

<code>IO_E_OK</code>	everything fine
<code>IO_E_INVALID_CHANNEL_ID</code>	the given channel id does not exist
<code>IO_E_FET_PROT_NOT_ACTIVE</code>	no output FET protection is active
<code>IO_E_FET_PROT_PERMANENT</code>	the output FET is protected permanently because it was already reset more than 10 times
<code>IO_E_FET_PROT_WAIT</code>	the output FET protection can not be reset, as the wait time of 10s is not already passed
<code>IO_E_CHANNEL_NOT_CONFIGURED</code>	the given channel is not configured
<code>IO_E_CH_CAPABILITY</code>	the given channel is not a PWM channel
<code>IO_E_INTERNAL_CSM</code>	an internal error occurred
<code>IO_E_UNKNOWN</code>	an unknown error occurred

Attention

The protection can be reset 10 times, afterwards the output will remain permanently protected

Note

- After entering the output protection, a certain time has to pass before the output protection can be reset:
 - 10s for `IO_PWM_00 .. IO_PWM_35`
- This function will set the output back again to the minimum pulse if a diagnostic margin is configured. After calling this function the output has to be set to the intended duty cycle with `IO_PWM_SetDuty()`

Remarks

If the parameter `reset_cnt` is **NULL**, the parameter is ignored. The parameter `reset_cnt` returns the number of resets already performed.

IO_ErrorType IO_PWM_ResolveOpenLoadShortCircuit (ubyte1 pwm_channel)

Resolve an open load or short circuit to battery voltage condition for a PWM channel.

Parameters

pwm_channel PWM channel, one of:

- **IO_PWM_00 .. IO_PWM_35**

Returns

IO_ErrorType

Return values

IO_E_OK	everything fine
IO_E_DRIVER_NOT_INITIALIZED	the common driver init function has not been called before
IO_E_BUSY	a previously started resolving operation is still ongoing
IO_E_INVALID_CHANNEL_ID	the given channel id does not exist
IO_E_CH_CAPABILITY	the given channel is not a PWM channel
IO_E_CHANNEL_NOT_CONFIGURED	the given channel is not configured
IO_E_INVALID_OPERATION	the given channel does not return IO_E_OPEN_LOAD_OR_SHORT_BAT or is not in an operational state

Attention

- The resolving of a PWM channel requires to shift an internal feedback signal that is used by all PWM channels. Although this does not influence other PWM channels, it can affect channels that are configured for the alternative functions digital input and timer input. Please refer to the safety manual for detailed information.
- The I/O driver does not provide any debouncing mechanism for the return value of function **IO_PWM_SetDuty()**. Thus, the application itself has to implement the debouncing mechanism before starting a resolving operation if required.

Note

- Resolving of a PWM channel can only be triggered if **IO_PWM_SetDuty()** returns **IO_E_OPEN_LOAD_OR_SHORT_BAT** for that channel.
- After resolving an open load or short circuit to battery condition the PWM channel will be automatically disabled and cannot be reenabled anymore.


```
IO_ErrorType IO_PWM_SetDuty ( ubyte1      pwm_channel,
                              ubyte2      duty_cycle,
                              ubyte2 *const high_time_fb,
                              ubyte2 *const period_fb
                              )
```

Set the duty cycle for a PWM channel.

Parameters

pwm_channel PWM channel, one of:

- `IO_PWM_00` .. `IO_PWM_35`

duty_cycle Duty cycle for the channel. Range: 0..65535 (0%..100%)

[out] **high_time_fb** High time feedback for the channels (optional). Returns high-time in us

[out] **period_fb** Period feedback for the channels (optional). Returns period in us

Returns

`IO_ErrorType`

Return values

<code>IO_E_OK</code>	everything fine
<code>IO_E_CHANNEL_NOT_CONFIGURED</code>	the given channel is not configured
<code>IO_E_INVALID_CHANNEL_ID</code>	the given channel id does not exist
<code>IO_E_CH_CAPABILITY</code>	the given channel has no PWM capability
<code>IO_E_DRIVER_NOT_INITIALIZED</code>	the common driver init function has not been called before
<code>IO_E_STARTUP</code>	the output is in the startup phase
<code>IO_E_PWM_NO_DIAG</code>	No diagnostic is possible currently. Either the set duty cycle is not within a diagnostic margin or not enough samples have been captured yet.
<code>IO_E_SHORT_GND</code>	short circuit has been detected
<code>IO_E_OPEN_LOAD_OR_SHORT_BAT</code>	open load or short circuit to battery has been detected
<code>IO_E_FET_PROT_ACTIVE</code>	the output FET protection is active and has set the output to a constant low level. It can be reset with <code>IO_PWM_ResetProtection()</code> after the wait time is passed
<code>IO_E_FET_PROT_REENABLE</code>	the output FET protection is ready to be reset with <code>IO_PWM_ResetProtection()</code>

IO_E_FET_PROT_PERMANENT	the output FET is protected permanently because it was already reset more than 10 times. The output will remain at a constant low level
IO_E_SAFETY_SWITCH_DISABLED	The safety switch of the corresponding output is disabled. The output is currently forced to low.
IO_E_SAFE_STATE	the PWM channel is in a safe state
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_UNKNOWN	an unknown error occurred
IO_E_RESOLVING	a previously started resolving operation for this channel is still ongoing
IO_E_RESOLVING_FAILED	a previously started resolving operation for this channel has failed to resolve the actual error (open load or short circuit to battery)

Remarks

- The duty cycle cannot exceed a margin of 100us(lower boundary) and 200us(upper boundary) used for diagnostic if the parameter `diag_margin` was set **TRUE** (via the function `IO_PWM_Init()`). This mode is important for hydraulic coils. If the parameter `diag_margin` is **FALSE**, no duty cycle range margin will be applied.
- If one of the parameter `high_time_fb` or `period_fb` is **NULL**, the parameter is ignored. The parameter `high_time_fb` returns the measured pulse-width (high time) of the PWM signal in the last round in us. The parameter `period_fb` returns the measured period of the PWM signal in the last round in us. If the measurement is not finished yet, the parameter `high_time_fb` or `period_fb` holds the value 0.

Attention

At the time when a PWM channel is initialized with `IO_PWM_Init()`, it has to be ensured that the actor is completely discharged and no current is induced to the PWM output. Otherwise the PWM current measurement calibration will fail.